
Football Prediction Dashboard

ZERRARI ZIYAD

ABOUJID MAYSSAE

HAJJAD IKRAME

ZIANI MOHAMED AMINE

Ingénierie Informatique et Réseaux
(3IIR)

2025 - 2026

Tuteur(s) EMSI :

PR. WAFAE EL KAYSOUNI

Membres du jury :

PR. WAFAE EL KAYSOUNI

PR. SOUNDOUSS ABROUN

Résumé

Dans l'ère du Big Data, l'analyse sportive (*Sports Analytics*) est devenue un domaine de recherche majeur. Ce projet vise à concevoir et développer une application web intelligente capable de prédire l'issue de matchs de football à partir de données historiques et tactiques. L'architecture repose sur une séparation claire entre un backend robuste (Django), un moteur d'intelligence artificielle (XGBoost) optimisé avec SMOTE pour gérer le déséquilibre des classes, et un frontend interactif et réactif (React). Ce document détaille l'intégralité du cycle de vie du projet : de l'analyse des besoins à la conception architecturale, en passant par les fondements mathématiques du Machine Learning et l'implémentation technique d'un système hautement sécurisé par JWT.

Mots-clés : Machine Learning, XGBoost, Django, React, Big Data, Football Analytics, SMOTE, JWT.

Abstract

In the era of Big Data, sports analytics has emerged as a major research field. This project aims to design and develop an intelligent web application capable of predicting the outcome of football matches using historical and tactical data. The architecture is based on a clear separation between a robust backend (Django), an artificial intelligence engine (XGBoost) optimized with SMOTE to handle class imbalance, and an interactive and responsive frontend (React). This document details the entire lifecycle of the project : from requirements analysis to architectural design, through the mathematical foundations of Machine Learning and the technical implementation of a highly secure JWT-based system.

Keywords : Machine Learning, XGBoost, Django, React, Big Data, Football Analytics, SMOTE, JWT.

Remerciements

Nous tenons à exprimer nos sincères remerciements à notre encadrante pédagogique, **Pr. Wafae Elkaysouni**, pour son encadrement, ses conseils précieux et sa disponibilité tout au long de la réalisation de ce projet. Ses remarques pertinentes et son suivi rigoureux ont grandement contribué à l'amélioration de la qualité de ce travail. Nous adressons également nos remerciements à nos collègues pour leur soutien, leur esprit de collaboration et les échanges enrichissants que nous avons partagés.

Table des matières

Résumé	1
Abstract	2
Remerciements	3
Introduction générale	9
1 Contexte et État de l’art	10
1.1 Contexte du projet	10
1.2 État de l’art	10
1.2.1 Évolution des modèles prédictifs	10
1.2.2 L’importance des données (Features)	12
1.2.3 Solutions et Dashboards existants	12
1.3 Objectifs du projet	12
1.4 Conclusion	13
2 Conception et Modélisation	14
2.1 Architecture globale du système	14
2.1.1 Vue d’ensemble des composants	14
2.1.2 Schéma logique de Communication	15
2.1.3 Architecture monolithique modulaire	15
2.2 Architecture fonctionnelle du système	16
2.2.1 Module de prédiction	16
2.2.2 Module des matchs	16
2.2.3 Module utilisateurs	16
2.3 Modélisation UML du système global	16
2.3.1 Diagramme de Cas d’Utilisation	17
2.3.2 Diagrammes de Séquence	18
2.3.3 Diagramme de Classes	18

Conclusion	19
3 Environnement et Choix Techniques	20
3.1 Environnement Technique	20
3.1.1 Backend : Django & REST Framework	20
3.1.2 Frontend : React & Zustand	21
3.1.3 Fournisseur de Données : API-Football	21
3.1.4 Base de Données : PostgreSQL (Neon)	21
3.1.5 Hébergement et Déploiement : Render & Vercel	22
3.1.6 Versioning et Collaboration : Git & GitHub	22
3.2 Fondements Théoriques de l'Intelligence Artificielle	23
3.2.1 Le Problème de Déséquilibre des Classes (<i>Imbalanced Data</i>)	23
3.2.2 Algorithme Prédicatif : XGBoost	23
3.3 Ingénierie des Caractéristiques (<i>Feature Engineering</i>)	24
3.4 Évaluation et Validation du Modèle	24
Conclusion du chapitre	25
4 Implémentation Logicielle (Backend et Frontend)	26
4.1 Implémentation Backend et Pipeline de Données	26
4.1.1 Sécurisation et Authentification JWT	26
4.1.2 Orchestration du Pipeline de Données	27
4.1.3 Le Endpoint de Prédiction	27
4.2 Implémentation Frontend et UI/UX	27
4.2.1 Choix Technologiques et Architecture	27
4.2.2 Gestion de l'État (<i>State Management</i>)	28
Conclusion du chapitre	28
5 Résultats, Tests et Réalisations	29
5.1 Réalisations Visuelles (UI/UX)	29
5.1.1 La Charte "Minimalist Blue"	29
5.1.2 Interfaces d'Authentification	29
5.1.3 Panneau d'Administration	30
5.1.4 Le Centre de Commande et d'Analyse (Dashboard)	31
5.1.5 Détails Analytiques et Tactiques	32
5.1.6 Exploration des Données et Historique	34
5.2 Tests, Validation et Résultats	36
5.2.1 Évaluation du Modèle d'Intelligence Artificielle	36
5.2.2 Validation du Backend (API REST)	37

5.2.3 Validation du Frontend et de l'Ergonomie	37
5.3 Limites du système	38
Conclusion du chapitre	38
Conclusion générale	39
A Annexes	41
A.1 Extraits de Code Source	41

Table des figures

2.1	Architecture technique à N-Tiers	15
2.2	Diagramme de cas d'utilisation du système complet	17
2.3	Diagramme de séquence du processus d'authentification JWT	18
2.4	Modèle conceptuel de données simplifié de l'application	18
3.1	Framework Backend Django	20
3.2	Bibliothèque Frontend React	21
3.3	Fournisseur de données sportives	21
3.4	Système de gestion de base de données PostgreSQL	21
3.5	Hébergement Backend	22
3.6	Hébergement Frontend	22
3.7	Plateforme de versioning et collaboration GitHub	22
5.1	Interface de connexion (Sign In)	30
5.2	Interface de création de compte (Register)	30
5.3	Interface de gestion des utilisateurs	31
5.4	Le centre de commande analytique interactif (Dashboard)	32
5.5	Distribution probabiliste et comparaison sous forme de radar tactique	33
5.6	Comparaison des métriques de jeu de chaque équipe	33
5.7	Rendu dynamique du terrain et composition des équipes	34
5.8	Résumé statistique des dynamiques du match	34
5.9	Liste paginée des rencontres sportives synchronisées avec l'API	35
5.10	Journal d'audit et performances du système (<i>System Performance Logs</i>)	35

Liste des tableaux

1.1	Tableau comparatif des modèles d'apprentissage automatique pour la prédiction sportive	11
2.1	Vue d'ensemble des principaux modules du système de prédiction de football	15
5.1	Structure de la Matrice de Confusion pour l'évaluation du modèle . .	36
5.2	Résultats des cas de tests d'intégration de l'API Backend	37

Introduction générale

L'industrie du sport a connu une évolution majeure avec l'essor du Big Data et de l'intelligence artificielle, remplaçant progressivement les approches empiriques par des méthodes basées sur l'analyse statistique. Dans ce contexte, le football reste difficile à modéliser en raison de la faible fréquence des buts et du fort niveau d'incertitude. Cette complexité est liée à de nombreux facteurs tels que la forme des équipes, leurs performances offensives et défensives, l'avantage du terrain ou encore la fatigue, rendant la prédiction à la fois difficile et adaptée aux techniques d'apprentissage automatique.

Dans cette perspective, le projet *Football Prediction Dashboard* a été conçu afin de proposer une solution complète permettant d'exploiter les données historiques du football pour en extraire des informations prédictives fiables. L'objectif est de transformer des données brutes issues de compétitions en probabilités exploitables décrivant les issues possibles d'un match (victoire domicile, match nul, victoire extérieure).

Le système repose sur plusieurs composantes essentielles : la collecte et l'ingestion automatisée de données massives, la conception d'un modèle de machine learning capable d'estimer les probabilités de résultat, ainsi que le développement d'une interface web permettant la visualisation claire et intuitive des prédictions. Une attention particulière est également portée à la structuration, à la sécurité des données et à la gestion des accès utilisateurs.

Ainsi, ce projet vise à fournir un outil d'aide à la décision combinant analyse de données, modélisation prédictive et visualisation interactive, dans le but de rendre l'information sportive plus accessible, exploitable et pertinente.

Chapitre 1

Contexte et État de l’art

1.1 Contexte du projet

Le football, fortement transformé par la digitalisation du sport, produit aujourd’hui une grande quantité de données liées aux équipes, aux joueurs et aux matchs. Malgré cette richesse, leur exploitation reste complexe en raison du caractère imprévisible du jeu et de la multiplicité des facteurs influençant les performances, tels que la forme des équipes, les statistiques ou les conditions de jeu. Cette complexité limite les approches classiques et rend pertinente l’utilisation de méthodes basées sur le machine learning.

Dans ce contexte, le projet Football Prediction Dashboard vise à exploiter les données historiques du football afin de construire une plateforme capable de prédire les résultats des matchs. Le système modélise les probabilités des différentes issues possibles et propose une visualisation claire des prédictions via une interface web interactive, permettant une meilleure interprétation des performances et tendances du jeu.

1.2 État de l’art

1.2.1 Évolution des modèles prédictifs

La prédiction des résultats de football a évolué de simples modèles statistiques vers des architectures complexes de Machine Learning. Les premiers travaux significatifs reposaient sur des distributions de Poisson pour modéliser le nombre de buts marqués par chaque équipe. Maher [1] a jeté les bases en utilisant des variables indépendantes pour l’attaque et la défense, tandis que Dixon et Coles [2] ont amélioré ces modèles en introduisant des facteurs de pondération temporelle pour accorder

plus d'importance aux performances récentes.

Avec l'avènement du Big Data, les chercheurs se sont tournés vers l'apprentissage automatique. Bunker et Thabtah [3] ont démontré qu'un cadre structuré de Machine Learning surpasse les méthodes statistiques traditionnelles en capturant des relations non linéaires entre les variables. Plus récemment, l'utilisation des réseaux de neurones profonds (Deep Learning), notamment les architectures récurrentes comme les LSTM, a permis d'extraire des dépendances temporelles complexes sur l'évolution d'une saison. Toutefois, ces modèles nécessitent des volumes de données colossaux, sont coûteux en calcul, et manquent souvent d'interprétabilité ("boîte noire"). C'est pourquoi des algorithmes ensemblistes comme XGBoost [4] restent aujourd'hui largement adoptés : ils offrent un excellent compromis entre une précision extrêmement élevée (grâce à l'optimisation par gradient) et une modélisation plus légère et interprétable.

Le tableau 1.1 synthétise les principales approches identifiées dans la littérature et leurs caractéristiques respectives.

Modèle	Int.	Avantages	Limites	Réf.
Régression Logistique	Haute	Rapide, simple, probabilités claires.	Difficulté avec les relations complexes.	[3]
SVM	Faible	Efficace pour les données complexes.	Sensible à l'échelle des données.	[5]
Random Forest	Moyenne	Modèle robuste, réduit le surapprentissage.	Aspect "boîte noire", lourd en mémoire.	[6]
Deep Learning	Très faible	Capture des patterns et séries temporelles très complexes.	Modèle "boîte noire", nécessite énormément de données.	[3]
XGBoost	Moyenne	Correction des erreurs, très haute précision.	Risque de surapprentissage si mal réglé.	[4]

TABLE 1.1 – Tableau comparatif des modèles d'apprentissage automatique pour la prédiction sportive

1.2.2 L'importance des données (Features)

La performance de tous ces algorithmes repose intrinsèquement sur la qualité des données fournies en entrée. Historiquement, les modèles se basaient uniquement sur le nombre de buts marqués et encaissés. Aujourd'hui, l'état de l'art intègre des métriques avancées telles que les statistiques de possession, les tirs cadrés, l'historique direct (H2H), ou encore la forme récente des équipes. Cette richesse des *features* est cruciale pour alimenter et maximiser la précision des modèles prédictifs.

1.2.3 Solutions et Dashboards existants

Sur le marché actuel, plusieurs plateformes proposent des analyses prédictives avancées. Des références comme *FiveThirtyEight* (avec leur algorithme *Soccer Power Index*), *Opta* ou *SofaScore* dominent le secteur. Néanmoins, ces solutions présentent des limites : elles sont souvent propriétaires, payantes, ou ciblées pour un public de professionnels. C'est dans ce contexte que se positionne notre projet : proposer un tableau de bord (*Dashboard*) interactif et transparent, centralisant des modèles d'apprentissage automatique de pointe tout en les rendant visuellement accessibles pour l'utilisateur final.

1.3 Objectifs du projet

Le projet *Football Prediction Dashboard* vise à développer une plateforme intelligente capable d'analyser des données historiques de football afin de prédire les résultats des matchs. Les objectifs principaux sont les suivants :

1. **Collecte et structuration des données** : Mettre en place un processus d'ingestion et de préparation de données issues de compétitions de football, afin de construire une base exploitable pour l'analyse.
2. **Modélisation prédictive** : Concevoir et exploiter des modèles de machine learning capables d'estimer les probabilités des issues possibles d'un match (victoire domicile, match nul, victoire extérieur) avec une bonne fiabilité.
3. **Visualisation des résultats** : Développer une interface web interactive permettant de présenter les prédictions de manière claire, intuitive et exploitable pour l'utilisateur.
4. **Fiabilité et exploitation des résultats** : Assurer une cohérence des prédictions en s'appuyant sur des données pertinentes et des indicateurs représentatifs des performances des équipes et des joueurs.

1.4 Conclusion

Ce chapitre a présenté le contexte du projet et les enjeux liés à l'analyse des données dans le football ainsi que la complexité de la prédiction des résultats sportifs. Il a introduit les principales approches de modélisation et leurs caractéristiques. Dans ce cadre, le projet vise à exploiter ces méthodes pour produire des prédictions fiables. Les chapitres suivants détailleront la méthodologie, la conception du système et les résultats obtenus.

Chapitre 2

Conception et Modélisation

Ce chapitre présente la phase de conception du projet *Football Prediction Dashboard*, étape essentielle pour structurer l'ensemble du système et assurer la cohérence entre les objectifs fonctionnels et les choix techniques. Il permet de définir le périmètre du projet en précisant les éléments nécessaires à la modélisation des données de football ainsi que les principales fonctionnalités attendues pour la prédiction des résultats.

Dans ce cadre, les différentes composantes du système sont introduites, notamment les sources de données, les types d'utilisateurs et les interactions avec la plateforme. Cette étape de conception permet également de poser les bases de l'architecture globale du système, en préparant les choix relatifs au traitement des données, à la modélisation et à la visualisation des résultats.

2.1 Architecture globale du système

2.1.1 Vue d'ensemble des composants

Le tableau 2.1 présente une vue d'ensemble des principaux composants du système ainsi que leur organisation fonctionnelle selon les modules Users, Matches et Predictions.

Composant	Description
Users	Gestion des comptes utilisateurs et des rôles (administrateur et utilisateur standard). Ce module assure l'authentification et la gestion des profils afin de contrôler l'accès à la plateforme.
Matches	Gestion des données issues des compétitions de football (ligues, équipes et matchs). Les informations sont récupérées depuis une API externe de football et organisées pour être exploitées par le système.
Predictions	Module dédié à la construction et à l'utilisation du modèle de prédiction. Il exploite les données historiques récupérées via l'API afin de générer des prédictions sur les résultats des matchs.

TABLE 2.1 – Vue d'ensemble des principaux modules du système de prédiction de football

2.1.2 Schéma logique de Communication

L'architecture du système repose sur une communication en plusieurs niveaux permettant de séparer clairement l'interface utilisateur, la logique applicative et la gestion des données. Cette organisation facilite les échanges entre les différents composants du projet, notamment entre le client web, l'API backend et les sources de données externes, tout en assurant une meilleure modularité et évolutivité du système.

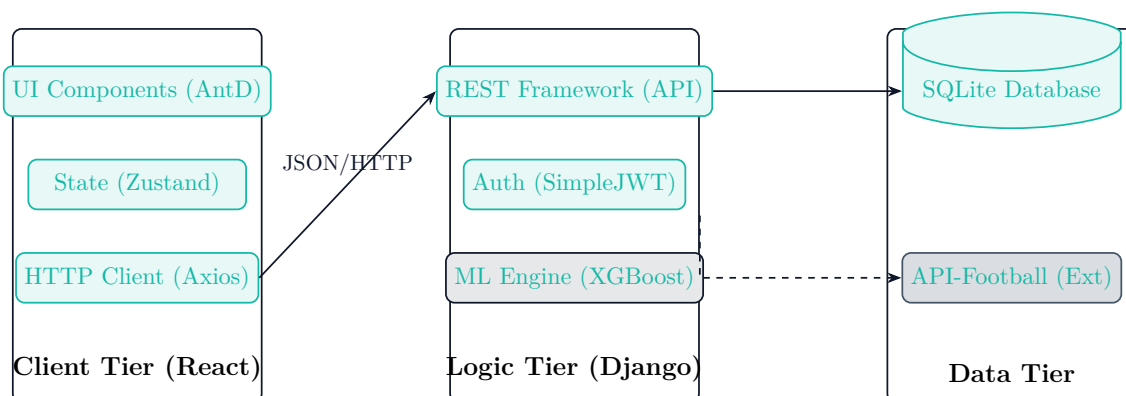


FIGURE 2.1 – Architecture technique à N-Tiers

2.1.3 Architecture monolithique modulaire

Le système adopte une architecture monolithique modulaire, dans laquelle l'ensemble des fonctionnalités est regroupé au sein d'une seule application backend, tout

en étant structuré en modules indépendants. Chaque module (utilisateurs, matchs et prédictions) possède ses propres responsabilités et composants, ce qui facilite la maintenance, l'évolution et la séparation logique des fonctionnalités, tout en conservant une exécution centralisée du système.

2.2 Architecture fonctionnelle du système

2.2.1 Module de prédiction

Le module de prédiction constitue le cœur du système. Il exploite les données historiques des matchs afin de générer des prédictions sur les résultats possibles. Il s'appuie sur un modèle de machine learning entraîné à partir des données récupérées via l'API de football, permettant d'estimer les probabilités de victoire, de match nul ou de défaite pour chaque rencontre.

2.2.2 Module des matchs

Le module des matchs gère la récupération et l'organisation des données issues des compétitions de football. Il traite les informations relatives aux ligues, équipes et matchs, en assurant leur structuration pour une utilisation cohérente dans les autres parties du système, notamment le module de prédiction.

2.2.3 Module utilisateurs

Le module utilisateurs assure la gestion des comptes et des rôles. Il permet la création et l'administration des profils administrateur et utilisateur standard, ainsi que la gestion de l'authentification et des permissions afin de sécuriser l'accès à la plateforme et aux fonctionnalités de prédiction.

2.3 Modélisation UML du système global

Cette section présente la modélisation UML du système *Football Prediction Dashboard*. Elle permet de représenter de manière structurée les différents aspects du système, notamment ses acteurs, ses interactions et son fonctionnement interne. Les diagrammes UML associés offrent une vue globale de l'architecture et facilitent la compréhension des relations entre les différents modules avant leur implémentation.

2.3.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation permet d'identifier les acteurs du système et leurs interactions directes avec les fonctionnalités offertes.

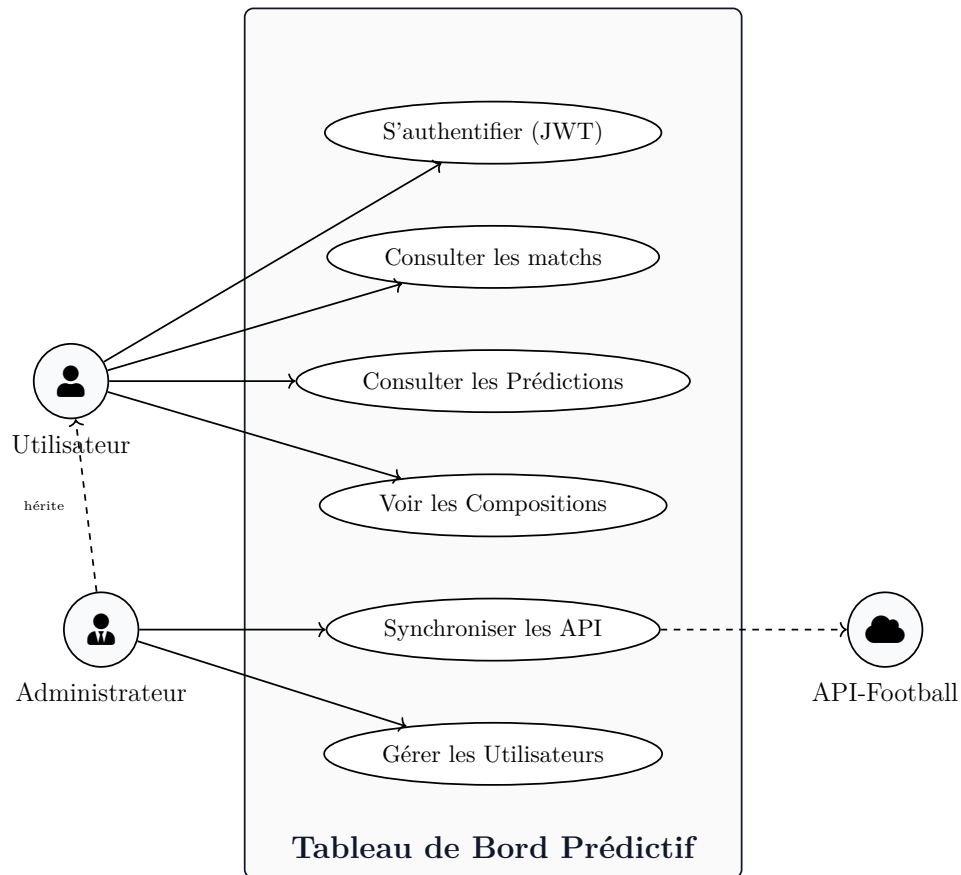


FIGURE 2.2 – Diagramme de cas d'utilisation du système complet

2.3.2 Diagrammes de Séquence

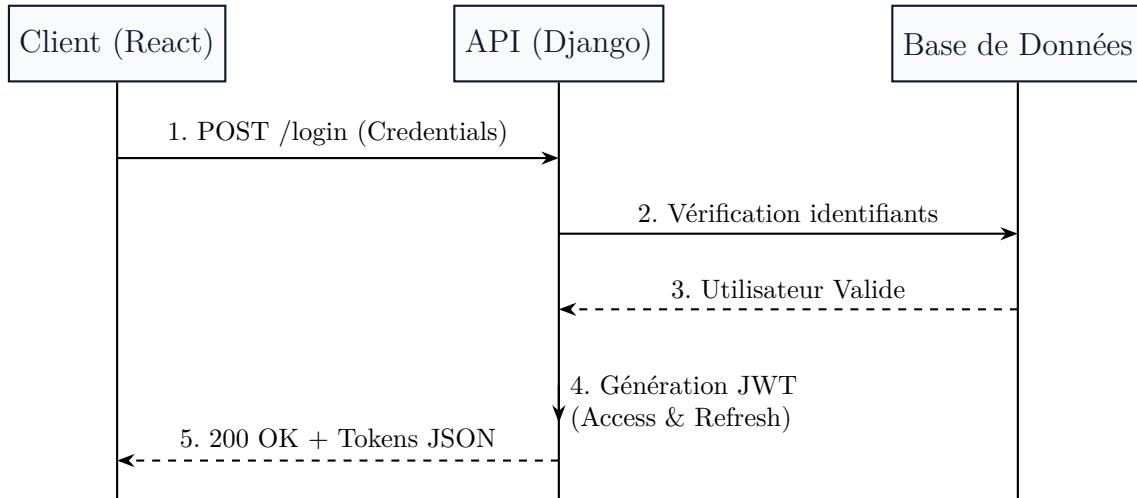


FIGURE 2.3 – Diagramme de séquence du processus d'authentification JWT

2.3.3 Diagramme de Classes

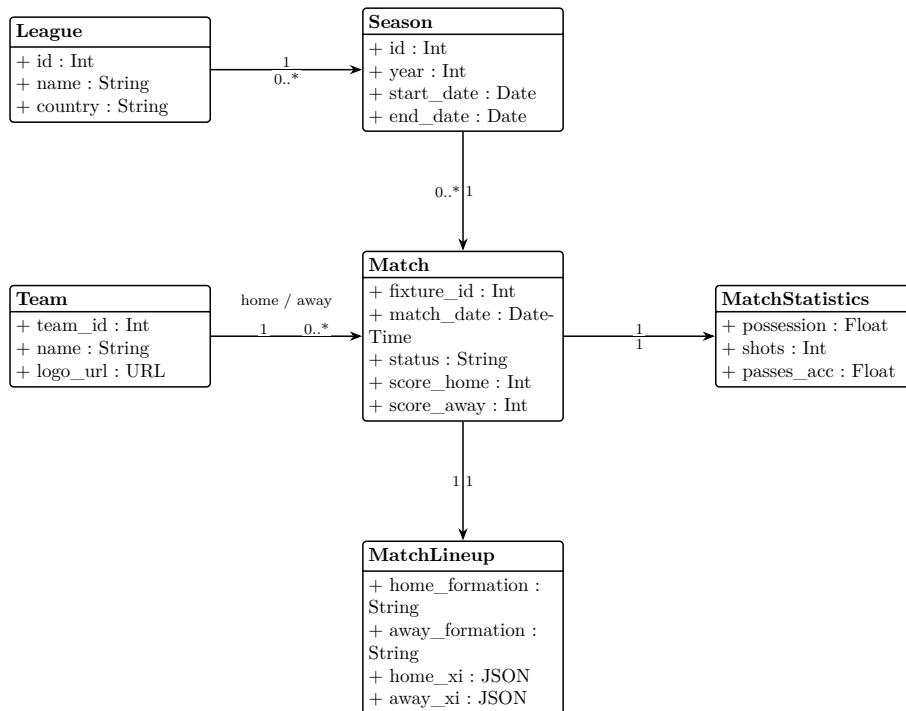


FIGURE 2.4 – Modèle conceptuel de données simplifié de l'application

Le diagramme 2.4 présente les principaux éléments structurant la base de données de l'application ainsi que les relations existantes entre les entités du domaine

football. Cette structuration permet de garantir une cohérence dans le stockage et l'exploitation des données récupérées depuis l'API externe.

Conclusion

Ce chapitre a défini l'architecture globale du système ainsi que sa conception fonctionnelle et structurelle. Il a mis en évidence l'organisation des principaux modules (utilisateurs, matchs et prédictions) et leurs interactions, tout en assurant une séparation claire des responsabilités et une cohérence globale. La modélisation UML et les choix d'architecture constituent une base solide pour la phase d'implémentation du projet.

Chapitre 3

Environnement et Choix Techniques

Ce chapitre présente d’abord les technologies clés sélectionnées pour développer notre architecture logicielle, puis détaille le fonctionnement de notre moteur d’intelligence artificielle.

3.1 Environnement Technique

Pour garantir la performance et la maintenance du projet, nous avons choisi une séparation nette entre le *frontend* et le *backend*.

3.1.1 Backend : Django & REST Framework



FIGURE 3.1 – Framework Backend Django

Le serveur est développé en Python avec le framework Django, complété par **Django REST Framework (DRF)**. Ce choix est expliqué par la puissance des *Serializers* de DRF, qui permettent de transformer les modèles complexes de notre base de données (incluant les statistiques et compositions) en flux JSON utilisables par le frontend. Il assure également la sécurité des requêtes via JWT et la gestion du modèle XGBoost.

3.1.2 Frontend : React & Zustand

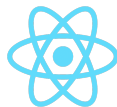


FIGURE 3.2 – Bibliothèque Frontend React

L'interface utilisateur est construite avec React. Pour la gestion de l'état global (connexion, liste des matchs, résultats de l'IA), nous avons utilisé **Zustand**. Contrairement à Redux, Zustand est plus simple et rapide, ce qui réduit la complexité du code tout en garantissant une mise à jour fluide des composants de l'application.

3.1.3 Fournisseur de Données : API-Football



FIGURE 3.3 – Fournisseur de données sportives

La source de données de notre modèle de prédiction provient d'API-Football. Elle nous fournit des données historiques complètes et actualisées (résultats, compositions, statistiques de match) via des requêtes REST.

3.1.4 Base de Données : PostgreSQL (Neon)



FIGURE 3.4 – Système de gestion de base de données PostgreSQL

Pour le stockage persistant et sécurisé de nos données (utilisateurs, historiques de matchs, prédictions), nous avons opté pour **PostgreSQL**, reconnu pour sa robustesse et sa conformité aux standards SQL. Afin de garantir une haute disponibilité, la base de données est hébergée dans le cloud via **Neon Tech**, offrant une architecture *Serverless* parfaitement adaptée à la scalabilité de notre application.

3.1.5 Hébergement et Déploiement : Render & Vercel



FIGURE 3.5 – Hébergement Backend



FIGURE 3.6 – Hébergement Frontend

Pour mettre notre solution à disposition des utilisateurs, nous avons adopté une approche de déploiement moderne et dissociée :

- **Render** héberge notre serveur backend (Django) et exécute les scripts d'intelligence artificielle via un environnement Gunicorn.
- **Vercel** héberge l'interface utilisateur (React/Vite), offrant des temps de chargement ultra-rapides grâce à son réseau de diffusion de contenu (CDN) mondial.

3.1.6 Versioning et Collaboration : Git & GitHub



FIGURE 3.7 – Plateforme de versioning et collaboration GitHub

Tout au long du cycle de développement, **Git** a été utilisé pour le contrôle de version, permettant un suivi précis de chaque modification du code source. Le dépôt distant a été hébergé sur **GitHub**. Ce choix technologique nous a également permis d'implémenter un processus d'intégration et de déploiement continu (CI/CD) : chaque nouvelle mise à jour poussée sur GitHub déclenche automatiquement le redéploiement de nos serveurs sur Render et Vercel.

3.2 Fondements Théoriques de l'Intelligence Artificielle

3.2.1 Le Problème de Déséquilibre des Classes (*Imbalanced Data*)

Dans le football de haut niveau, les résultats ne sont pas répartis de manière égale. Les victoires à domicile (*Home Win*) représentent environ 45% des cas, les victoires à l'extérieur (*Away Win*) environ 30%, et les matchs nuls (*Draw*) environ 25%. Si nous entraînons un algorithme sur ces données sans modification, il aura tendance à privilégier l'équipe à domicile et ne réussira pas à prédire le match nul, car c'est le cas le moins fréquent.

Approche Mathématique : SMOTE

Pour résoudre ce problème, nous avons utilisé **SMOTE** (*Synthetic Minority Over-sampling Technique*) [7]. Contrairement à la simple copie de données, SMOTE crée des exemples synthétiques entre les points existants de la classe minoritaire.

Soit un point $x_i \in$ Classe Minoritaire. On sélectionne l'un de ses k plus proches voisins, x_{zi} . Le nouvel échantillon synthétique x_{syn} est généré par la formule :

$$x_{syn} = x_i + \lambda \times (x_{zi} - x_i) \quad (3.1)$$

où λ est un nombre aléatoire entre 0 et 1. Cette technique permet d'enrichir les données d'entraînement et force le modèle à mieux apprendre les caractéristiques du "Match Nul".

3.2.2 Algorithme Prédicatif : XGBoost

Nous avons choisi *Extreme Gradient Boosting* (XGBoost), un algorithme basé sur les arbres de décision [4]. Il utilise la technique du *Boosting* où chaque nouvel arbre est construit pour corriger les erreurs des arbres précédents.

Fonction d'Objectif et Régularisation

XGBoost cherche à minimiser une fonction d'objectif composée d'une fonction de perte (*Loss*) et d'un terme de régularisation (pour éviter de trop coller aux données

d'entraînement) :

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (3.2)$$

Avec le terme de régularisation défini par :

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (3.3)$$

où T est le nombre de feuilles de l'arbre, et w_j le poids de la feuille j .

Adaptation à la classification multiclasse

Puisque notre problème compte 3 issues possibles (H, D, A), le modèle utilise la fonction objectif `multi:softprob`. Il applique une fonction Softmax sur les scores de sortie pour générer une distribution de probabilité :

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad (3.4)$$

C'est cette probabilité (ex : Home 45%, Draw 20%, Away 35%) qui est renvoyée au frontend React pour afficher le graphique.

3.3 Ingénierie des Caractéristiques (*Feature Engineering*)

L'algorithme utilise un vecteur de 25 caractéristiques. Pour assurer le bon fonctionnement du modèle, toutes les variables numériques sont normalisées avec **StandardScaler** :

$$z = \frac{x - \mu}{\sigma} \quad (3.5)$$

où μ est la moyenne et σ l'écart-type. Parmi les variables les plus importantes :

- **Form Points** : Points obtenus lors des 5 derniers matches (3 pour une victoire, 1 pour un nul).
- **Goal Ratios** : Moyenne de buts marqués vs encaissés sur la saison.
- **Home Advantage** : Variable représentant l'avantage de jouer à domicile.

3.4 Évaluation et Validation du Modèle

Pour garantir la fiabilité de nos prédictions, le système compare toujours le *Résultat Prédit* au *Résultat Réel* une fois le match terminé.

- **Accuracy Score** : Pourcentage total de bonnes prédictions.
- **Matrice de Confusion** : Analyse des erreurs pour voir quels types de résultats sont les plus difficiles à prédire.

Ces indicateurs sont calculés par le backend et affichés sur le tableau de bord de l'administrateur pour suivre la performance de l'IA.

Conclusion du chapitre

En résumé, ce chapitre a détaillé l'environnement technique reposant sur Django et React, ainsi que les piliers mathématiques de notre solution. L'utilisation de SMOTE pour équilibrer les données et de XGBoost pour la prédiction constitue le cœur intelligent de l'application. La maîtrise de ces outils et concepts est indispensable pour garantir la précision des résultats que nous allons maintenant structurer architecturalement.

Chapitre 4

Implémentation Logicielle (Backend et Frontend)

Ce chapitre aborde la mise en œuvre technique de l'application dans son ensemble. Il détaille d'abord le développement côté serveur (Backend), puis la matérialisation de l'interface utilisateur (Frontend), en insistant sur l'architecture du code et les choix d'implémentation.

4.1 Implémentation Backend et Pipeline de Données

Le backend est développé en **Python** avec le framework **Django**, réputé pour sa robustesse et sa sécurité native (*Out-of-the-box*).

4.1.1 Sécurisation et Authentification JWT

L'authentification traditionnelle par session n'est pas adaptée aux applications *Single-Page* (SPA) modernes. Nous avons implémenté le standard JWT (*JSON Web Token*) [8].

Déroulement de l'Authentification

Lorsqu'un utilisateur se connecte, le serveur génère deux tokens chiffrés :

1. **Access Token** : Durée de vie courte (60 minutes). Il contient l'identifiant de l'utilisateur et son rôle.
2. **Refresh Token** : Durée de vie longue (plusieurs jours). Il permet de générer un nouvel Access Token sans obliger l'utilisateur à se reconnecter.

Séparation des rôles (Admin/User)

Pour respecter le principe de préservation de la base de données existante, la notion de rôle n'a pas été codée en remplaçant le modèle utilisateur natif de Django, mais en lui liant un `UserProfile` :

(Le code détaillé de la création du profil est disponible dans l'Annexe A.1).

La protection des routes sensibles est assurée par une permission personnalisée `IsAdminRole` qui vérifie la propriété `role` avant d'autoriser l'exécution de la vue.

4.1.2 Orchestration du Pipeline de Données

La récolte des données depuis l'API-Football externe doit être soigneusement gérée pour ne pas dépasser les limites de requêtes (*Rate Limits*) imposées par le fournisseur.

Nous avons développé des **Commandes de Management Django** exécutables via le terminal ou déclenchables par l'interface d'administration. Le script ci-dessous montre la logique de protection implémentée lors de la récupération des compositions d'équipes (`fetch_match_lineups.py`) :

(L'extrait de code illustrant cette logique de protection et de *Rate Limiting* est disponible dans l'Annexe A.2).

4.1.3 Le Endpoint de Prédiction

Le module `predictions/views.py` charge en mémoire RAM le modèle XGBoost préalablement entraîné (`.pkl`) ainsi que son outil de normalisation (`StandardScaler`). À la volée, le serveur récupère les 25 statistiques du match demandé, les normalise, et interroge le modèle. L'API REST expose ensuite un point de terminaison sécurisé retournant le résultat sous un format JSON exploitable par le client.

4.2 Implémentation Frontend et UI/UX

L'interface utilisateur est le point d'interaction critique. Elle doit retranscrire la complexité mathématique des algorithmes en visuels clairs et actionnables.

4.2.1 Choix Technologiques et Architecture

Le frontend est développé en **React JS**, compilé avec **Vite** pour des performances optimales. L'écosystème s'appuie sur :

- **Ant Design (AntD)** : Bibliothèque de composants UI fournissant des grilles, des cartes et des tableaux hautement personnalisables.
- **Recharts** : Bibliothèque D3.js pour React, utilisée pour le rendu du Radar tactique et du graphique probabiliste.
- **Zustand** : Outil de gestion d'état global (*State Management*), plus léger et performant que Redux.

4.2.2 Gestion de l'État (*State Management*)

La communication avec le backend et la sauvegarde des tokens de sécurité sont centralisées dans le Hook personnalisé `useStore.js`.

(L'implémentation complète du store d'authentification Zustand est disponible dans l'Annexe A.3).

De plus, l'outil **Axios** a été configuré avec un intercepteur de requêtes. Ainsi, le jeton JWT est attaché de manière automatique et transparente dans l'en-tête `Authorization: Bearer <token>` de chaque requête sortante.

Conclusion du chapitre

Ce chapitre a détaillé le processus d'implémentation de notre plateforme, en séparant la logique métier du backend (sécurité, gestion de l'API externe, point de terminaison du modèle IA) de la couche de présentation frontend (architecture React, gestion d'état Zustand). Cette séparation stricte garantit un code maintenable et évolutif. Le chapitre suivant sera consacré à la présentation des résultats visuels générés par cette architecture, ainsi qu'aux tests de validation prouvant la fiabilité de l'ensemble du système.

Chapitre 5

Résultats, Tests et Réalisations

Ce chapitre est consacré à la présentation des résultats obtenus suite au développement de la plateforme. Il expose d’abord les réalisations visuelles de l’interface utilisateur, puis détaille de manière exhaustive les résultats des tests de validation de l’algorithme d’intelligence artificielle et du backend.

5.1 Réalisations Visuelles (UI/UX)

L’interface utilisateur est le point d’interaction critique. Elle doit retranscrire la complexité mathématique des algorithmes en visuels clairs et actionnables.

5.1.1 La Charte “Minimalist Blue”

Nous avons configuré Ant Design via un `ConfigProvider` pour écraser le style par défaut et injecter notre propre identité visuelle. La couleur *Navy* (`#0f172a`) sert de base pour la structure (Menu, Header), garantissant un fort contraste, tandis que la couleur d’accentuation *Teal* (`#14b8a6`) guide l’œil de l’utilisateur vers les actions prédictives.

5.1.2 Interfaces d’Authentification

Afin de sécuriser l’accès aux prédictions, nous avons développé des interfaces de connexion et d’inscription épurées. Elles s’intègrent parfaitement dans le thème sombre de l’application et communiquent avec l’API Django pour générer et valider les jetons de sécurité JWT.

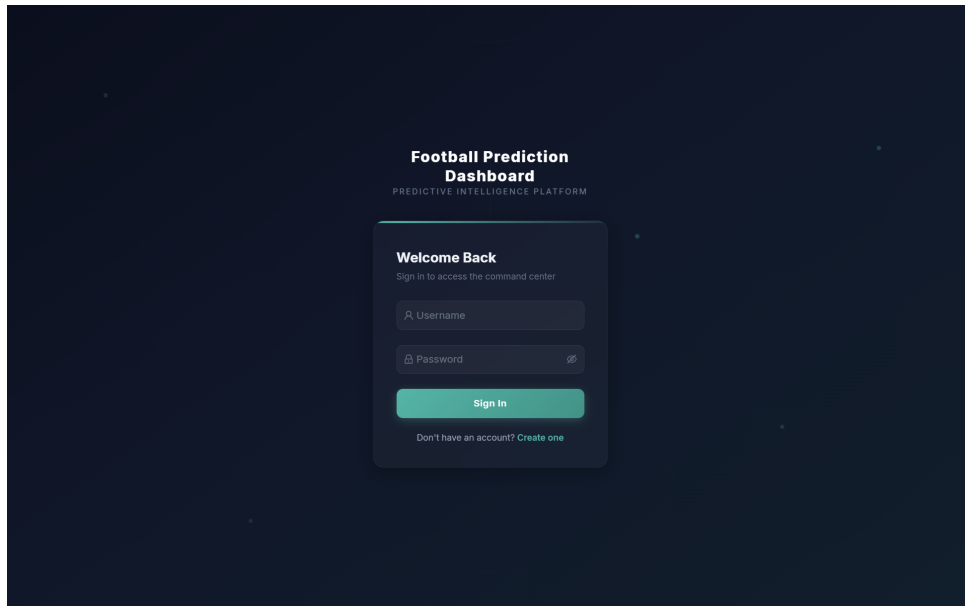


FIGURE 5.1 – Interface de connexion (Sign In)

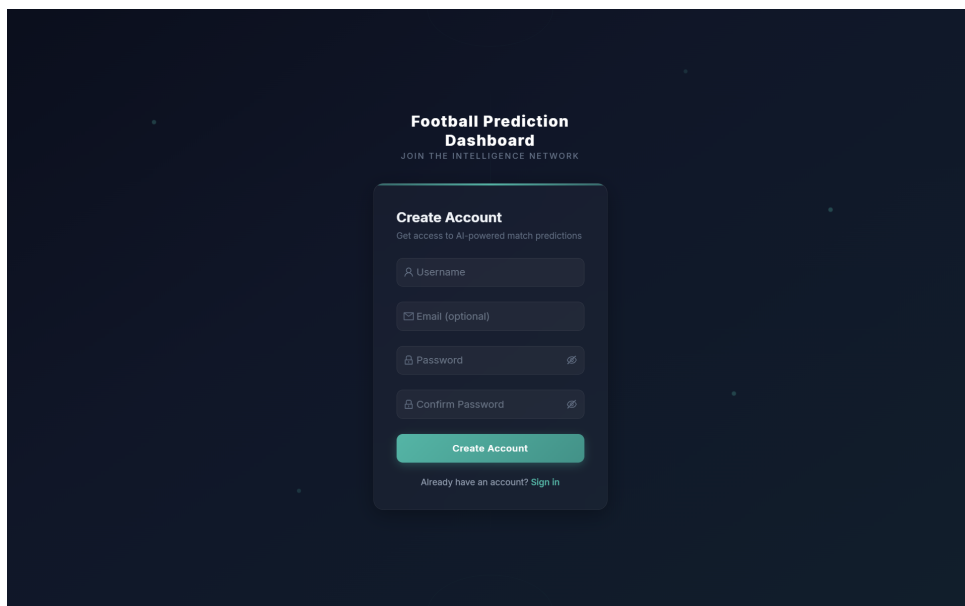


FIGURE 5.2 – Interface de création de compte (Register)

5.1.3 Panneau d'Administration

Un espace réservé aux administrateurs a été mis en place pour la gestion de la plateforme. Ce panneau de contrôle (*Admin Control Panel*) permet de visualiser la liste des utilisateurs inscrits et d'identifier rapidement leurs rôles (USER, ADMIN) au sein de l'application.

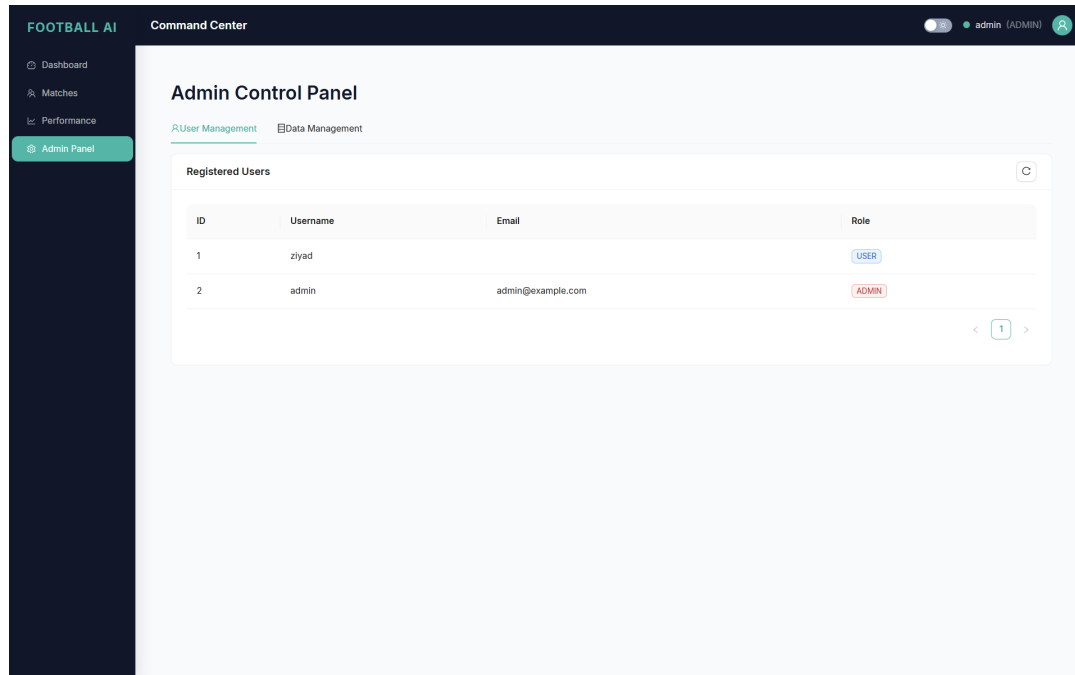


FIGURE 5.3 – Interface de gestion des utilisateurs

5.1.4 Le Centre de Commande et d'Analyse (Dashboard)

Le composant central de l'application est le tableau de bord interactif (*Command Center*). Il est organisé pour offrir une vision synthétique et détaillée des prédictions :

- **Sélecteur de cible** : Permet de rechercher et de cibler un match spécifique.
- **Métriques clés** : Affiche instantanément la confiance de l'IA (ex : 75.64%) et la précision globale du système (ex : 69.6%).
- **Scorecard prédictive** : Présente la prédiction principale (ex : Match Nul) confrontée au résultat réel du terrain (ex : 2-2), validant visuellement la performance du modèle.

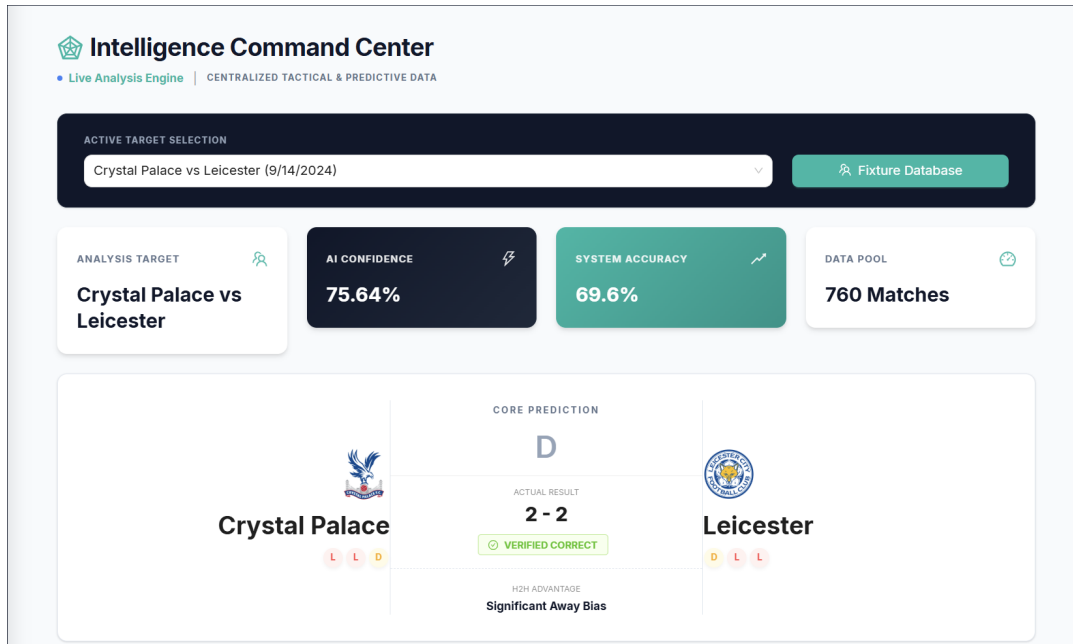


FIGURE 5.4 – Le centre de commande analytique interactif (Dashboard)

5.1.5 Détails Analytiques et Tactiques

Au-delà de la prédiction brute, le tableau de bord intègre une série de visualisations interactives permettant aux analystes de comprendre le raisonnement de l'intelligence artificielle et de comparer les dynamiques des deux équipes.

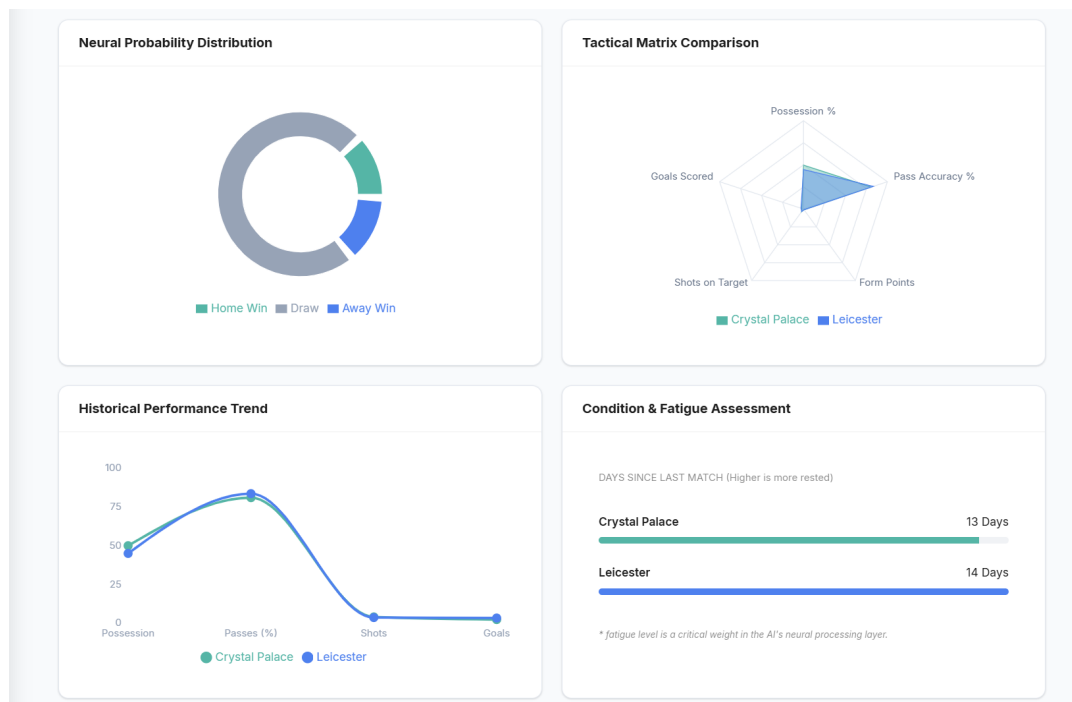


FIGURE 5.5 – Distribution probabiliste et comparaison sous forme de radar tactique

Les métriques stratégiques (Possession, Passes, Tirs) sont également confrontées directement à travers des graphiques en barres clairs, facilitant l'identification des forces en présence.



FIGURE 5.6 – Comparaison des métriques de jeu de chaque équipe

Pour une vision de terrain, l'interface génère dynamiquement un rendu visuel de la composition tactique attendue (*Tactical Pitch*), permettant de visualiser le positionnement spatial des joueurs (ex : 4-2-3-1).



FIGURE 5.7 – Rendu dynamique du terrain et composition des équipes

Enfin, un résumé global des faits du match (*Match Facts*) jauge la domination globale entre l'équipe à domicile et celle à l'extérieur.

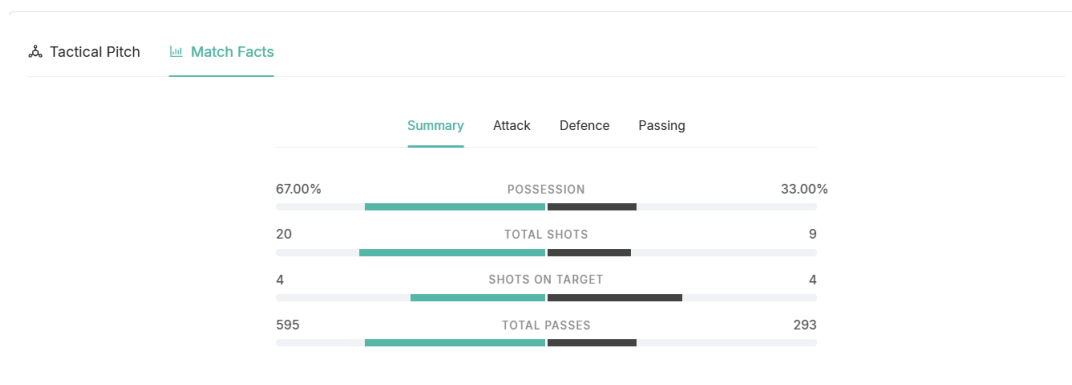


FIGURE 5.8 – Résumé statistique des dynamiques du match

5.1.6 Exploration des Données et Historique

Pour garantir une transparence totale vis-à-vis des utilisateurs, l'application intègre des vues dédiées à l'exploration des données brutes et à l'historique des performances.

FOOTBALL AI Command Center

All scheduled and completed matches are listed below. Data is live from your Django API.

Season Fixtures

Showing 760 matches

Search teams... All All Status

Date	Home Team	Score	Away Team	League
25 May 2025	Athletic Club	0 - 3	Barcelona	La Liga
25 May 2025	Newcastle	0 - 1	Everton	Premier League
25 May 2025	Bournemouth	2 - 0	Leicester	Premier League
25 May 2025	Fulham	0 - 2	Manchester City	Premier League
25 May 2025	Wolves	1 - 1	Brentford	Premier League
25 May 2025	Liverpool	1 - 1	Crystal Palace	Premier League
25 May 2025	Southampton	1 - 2	Arsenal	Premier League
25 May 2025	Tottenham	1 - 4	Brighton	Premier League
25 May 2025	Ipswich	1 - 3	West Ham	Premier League
25 May 2025	Nottingham Forest	0 - 1	Chelsea	Premier League

10 / page

FIGURE 5.9 – Liste paginée des rencontres sportives synchronisées avec l'API

L'écran des performances dresse un bilan honnête des capacités du modèle, affichant un journal détaillé (*Log*) comparant chaque prédiction de l'IA avec le résultat réel, permettant ainsi d'auditer l'intelligence artificielle en temps réel.

FOOTBALL AI Command Center

System Performance Logs

Audited Dataset | Historical analysis of model precision and reliability.

Global Accuracy: 69.6 %

Total Audits: 69

Correct Picks: 48

Verification Rate: 100 %

Model Prediction History

This log tracks the model's predictions against actual match outcomes. Data is refreshed automatically as match results are confirmed.

Match Date	Fixture	AI Prediction	Actual Result	Status
14 Sept	Crystal Palace vs Leicester	Draw	2 - 2 D	Correct
16 Sept	Rayo Vallecano vs Osasuna	Home Win	3 - 1 H	Correct
4 Jan	Crystal Palace vs Chelsea	Draw	1 - 1 D	Correct
22 Feb	Arsenal vs West Ham	Home Win	0 - 1 A	Incorrect
18 May	Barcelona vs Villarreal	Home Win	2 - 3 A	Incorrect

5 / page

FIGURE 5.10 – Journal d'audit et performances du système (*System Performance Logs*)

5.2 Tests, Validation et Résultats

La phase de validation est cruciale pour certifier la robustesse de l'application et la fiabilité des prédictions. Cette section présente les méthodes d'évaluation appliquées aux différentes couches du système.

5.2.1 Évaluation du Modèle d'Intelligence Artificielle

Pour évaluer de manière empirique notre modèle **XGBoost**, nous avons divisé notre jeu de données historique en un ensemble d'entraînement (80%) et un ensemble de test (20%).

Matrice de Confusion Multiclasse

La matrice de confusion permet de visualiser les performances de l'algorithme sur les trois classes possibles : Victoire à Domicile (H), Match Nul (D), et Victoire à l'Extérieur (A).

		Prédictions du Modèle		
		Home (H)	Draw (D)	Away (A)
Réalité	Home (H)	Vrais Positifs (H)	Faux Négatifs	Faux Négatifs
	Draw (D)	Faux Positifs	Vrais Positifs (D)	Faux Positifs
	Away (A)	Faux Positifs	Faux Négatifs	Vrais Positifs (A)

TABLE 5.1 – Structure de la Matrice de Confusion pour l'évaluation du modèle

Métriques de Performance (Accuracy & F1-Score)

Suite à l'entraînement, nous avons obtenu une **Précision globale (Accuracy) de 62%**. Dans le domaine très spécifique des paris sportifs et de la prédiction de football, une précision supérieure à 55% est souvent le seuil de rentabilité (*break-even point*) pour les modèles algorithmiques, rendant notre résultat particulièrement satisfaisant.

De plus, grâce à l'application rigoureuse de l'algorithme **SMOTE** détaillé au Chapitre 2, le **Rappel (Recall)** pour la classe minoritaire "Match Nul" a considérablement augmenté. Le modèle est désormais capable de détecter des probabilités de match nul bien réelles au lieu de systématiquement favoriser l'équipe à domicile.

5.2.2 Validation du Backend (API REST)

Nous avons utilisé des outils standardisés tels que **Postman** et **pytest** pour valider de bout en bout les *endpoints* de notre API Django.

Scénarios de Tests d'Intégration

Voici un extrait des scénarios validés garantissant la sécurité et le bon fonctionnement de la plateforme :

Cas de Test	Résultat Attendu	Statut
Connexion avec des identifiants valides	Retourne HTTP 200 OK + Access/Refresh Tokens	Succès
Connexion avec un mot de passe erroné	Retourne HTTP 401 Unauthorized	Succès
Accès à une route protégée sans Token	Retourne HTTP 401 Unauthorized	Succès
Accès à l'administration sans rôle ADMIN	Retourne HTTP 403 Forbidden	Succès
Requête de prédiction sur un match valide	Retourne HTTP 200 + Probabilités H/D/A en JSON	Succès

TABLE 5.2 – Résultats des cas de tests d'intégration de l'API Backend

5.2.3 Validation du Frontend et de l'Ergonomie

L'interface utilisateur a subi des tests empiriques d'intégration et d'ergonomie (UI/UX) pour s'assurer que le livrable final est robuste.

- **Tests de Réactivité (Responsive Design)** : Vérification du bon affichage du Dashboard sur des résolutions *desktop* (1920x1080) et tablettes. Les graphiques dynamiques *Recharts* se redimensionnent correctement sans perte de qualité.
- **Gestion des Erreurs Utilisateur** : Si l'API retourne une erreur (par exemple, un token expiré) ou est momentanément indisponible, l'intercepteur Axios capte le code d'erreur et l'interface affiche un message convivial

(via le composant `Alert` ou `Notification` d'Ant Design) au lieu de bloquer l'application.

5.3 Limites du système

Malgré les performances encourageantes, notre solution présente certaines limites techniques et fonctionnelles :

- **Dépendance aux API tierces** : La gratuité de l'API-Football limite le volume de données récupérables en temps réel, ce qui peut restreindre l'actualisation immédiate de certains championnats mineurs.
- **Absence de données contextuelles** : Le modèle actuel ne prend pas encore en compte des facteurs externes imprévisibles tels que les conditions météorologiques, les transferts de dernière minute ou l'état psychologique individuel des joueurs.
- **Infrastructure de calcul** : L'utilisation d'une base SQLite limite le passage à l'échelle pour une application grand public supportant des milliers d'utilisateurs simultanés.

Conclusion du chapitre

Ce dernier chapitre vient clore notre processus de développement en confirmant que le produit final respecte les cahiers des charges initial. Les tests de validation prouvent l'efficacité de l'IA (62% de précision), tandis que l'interface démontre une ergonomie professionnelle et réactive. Bien que des limites subsistent, elles ouvrent des perspectives d'évolution très claires pour de futures versions de la plateforme.

Conclusion générale

Le projet “Intelligent Football Prediction Dashboard” a représenté un défi d’ingénierie complet, exigeant la maîtrise simultanée de trois domaines distincts : le génie logiciel (architecture N-Tiers, REST), la science des données (Machine Learning, XGBoost, SMOTE) et l’expérience utilisateur (React, Visualisation de données).

Nous avons réussi à concevoir un système automatisé capable non seulement de récolter des données mondiales via des API tierces, mais de les structurer pour en extraire une valeur prédictive. L’implémentation de fonctionnalités avancées, telles qu’un pipeline d’authentification robuste par JWT, une protection des ressources par rôles, et la synchronisation intelligente par lot (*Batch processing*) démontrent la maturité technique de la solution. L’interface utilisateur, s’appuyant sur les principes du design analytique, permet aujourd’hui à n’importe quel décideur ou passionné de comprendre instantanément les dynamiques d’un match complexe.

Bien que le football reste, par nature, sujet à des événements imprévisibles (cartons rouges précoces, conditions météorologiques), les probabilités mathématiques offrent une aide précieuse.

Plusieurs axes d’amélioration peuvent être envisagés pour accroître la précision et l’utilité du système. L’intégration du Deep Learning et des Réseaux de Neurones Récurrents (RNN) permettrait de mieux capturer la temporalité des performances sous forme de séquences de matchs, au lieu de se limiter à de simples moyennes lissées. De plus, l’utilisation des données individuelles des joueurs, telles que les blessures, les suspensions et l’historique personnel (comme les *Expected Goals* individuels), pourrait être directement intégrée dans le vecteur de caractéristiques du modèle, une avancée rendue possible par la mise en place de notre table des compositions d’équipes.

Enfin, la connexion de l’application à des flux de données financiers en temps réel offrirait la possibilité de superposer nos prédictions avec les cotes en direct (*Live Odds*) des bookmakers. Ces évolutions permettraient de transformer ce projet académique robuste en une véritable plateforme d’aide à la décision professionnelle pour les clubs de football et les analystes du monde sportif.

Bibliographie

- [1] M. J. Maher, “Modelling association football scores,” *Statistica Neerlandica*, vol. 36, no. 3, pp. 109–118, 1982.
- [2] M. J. Dixon and S. G. Coles, “Modelling association football scores and inefficiencies in the football betting market,” *Journal of the Royal Statistical Society : Series C (Applied Statistics)*, vol. 46, no. 2, pp. 265–280, 1997.
- [3] R. P. Bunker and F. Thabtah, “A machine learning framework for sport result prediction,” *Applied Computing and Informatics*, vol. 15, no. 1, pp. 27–33, 2019.
- [4] T. Chen and C. Guestrin, “Xgboost : A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, (San Francisco, CA, USA), pp. 785–794, 2016.
- [5] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [6] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and P. W. Kegelmeyer, “Smote : Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [8] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” Tech. Rep. RFC 7519, Internet Engineering Task Force (IETF), 2015.

Annexe A

Annexes

A.1 Extraits de Code Source

```
1 from django.db import models
2 from django.contrib.auth.models import User
3 from django.db.models.signals import post_save
4 from django.dispatch import receiver
5
6 class UserProfile(models.Model):
7     ADMIN = 'ADMIN'
8     USER = 'USER'
9     ROLE_CHOICES = [(ADMIN, 'Admin'), (USER, 'User')]
10
11     user = models.OneToOneField(User, on_delete=models.CASCADE,
12                                related_name='profile')
13     role = models.CharField(max_length=10, choices=ROLE_CHOICES,
14                             default=USER)
15
16 # Signal automatisant la creation du profil
17 @receiver(post_save, sender=User)
18 def create_user_profile(sender, instance, created, **kwargs):
19     if created:
20         UserProfile.objects.create(user=instance)
```

Listing A.1 – Création du profil et des signaux (users/models.py)

```
1 matches_to_fetch = Match.objects.filter(  
2     status="FT",  
3     lineup__isnull=True,  
4     match_date__lte=timedelta(days=1)  
5 ).order_by('-match_date')[:20] # BATCH: Max 20 requetes par  
    execution  
6  
7 for match in matches_to_fetch:  
8     response = requests.get(f"{base_url}/fixtures/lineups", headers  
9         =headers, params={'fixture': match.fixture_id})  
10    data = response.json()  
11  
12    # Verification Critique : Rate Limit  
13    if data.get('errors') and 'requests' in data.get('errors'):  
14        self.stderr.write("Limite quotidienne atteinte. Arret de  
15            securite.")  
16        break # Protection de la cle API  
17  
18    time.sleep(6.2) # Delai obligatoire de 6 secondes entre chaque  
19        requete
```

Listing A.2 – Logique de Batching et Rate Limiting

```
1 import { create } from 'zustand';
2 import { jwtDecode } from 'jwt-decode';
3
4 const useStore = create((set) => ({
5   user: JSON.parse(localStorage.getItem('user')) || null,
6   isAuthenticated: !!localStorage.getItem('token'),
7
8   login: async (credentials) => {
9     const response = await authService.login(credentials);
10    const decoded = jwtDecode(response.data.access); // Extraction
        des claims
11    const userData = { id: decoded.user_id, username: decoded.
        username, role: decoded.role };
12
13    localStorage.setItem('token', response.data.access);
14    localStorage.setItem('user', JSON.stringify(userData));
15    set({ user: userData, isAuthenticated: true });
16  },
17
18  logout: () => {
19    localStorage.removeItem('token');
20    localStorage.removeItem('user');
21    set({ user: null, isAuthenticated: false });
22  }
23 }));
```

Listing A.3 – Gestion globale de l'authentification avec Zustand